

Group Anagrams:-

Code:-

```
import java.util.*;

class Solution {

    public List<List<String>> groupAnagrams(String[] strs) {

        HashMap<String, List<String>> map = new HashMap<>();

        for (String str : strs) {

            char[] chars = str.toCharArray();

            Arrays.sort(chars);

            String key = new String(chars);

            if (!map.containsKey(key)) {

                map.put(key, new ArrayList<>());

            }

            map.get(key).add(str);

        }

        return new ArrayList<>(map.values());

    }

}
```

Output:-

The screenshot displays the LeetCode interface for the 'Group Anagrams' problem. The top section shows the problem description and a list of submissions. The submission by 'tameem_shaik' is highlighted, showing it was accepted on September 15, 2026, at 12:20. Below this, a performance comparison chart shows the submission's runtime (7 ms) and memory usage (49.87 MB) compared to other submissions. The bottom section shows the code editor with the Java solution, the test result, and the input/output for the test cases.

Code:

```
11         }
12         map.get(key).add(str);
13     }
14     return new ArrayList<>(map.values());
15 }
16 }
```

Test Result:

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

```
strs =
["eat","tea","tan","ate","nat","bat"]
```

Output

```
[["eat","tea","ate"],["bat"],["tan","nat"]]
```

Expected

```
[["bat"],["nat","tan"],["ate","eat","tea"]]
```

Contribute a testcase

Find the Highest Altitude:-

Code:-

```
import java.util.*;

class Solution {

    public int[] topKFrequent(int[] nums, int k) {

        HashMap<Integer, Integer> map = new HashMap<>();

        for (int num : nums) {

            map.put(num, map.getDefault(num, 0) + 1);

        }

        List<Integer>[] buckets = new List[nums.length + 1];

        for (int num : map.keySet()) {

            int frequency = map.get(num);

            if (buckets[frequency] == null) {

                buckets[frequency] = new ArrayList<>();

            }

            buckets[frequency].add(num);

        }

        int[] result = new int[k];

        int index = 0;

        for (int i = buckets.length - 1; i >= 0 && index < k; i--) {

            if (buckets[i] != null) {

                for (int num : buckets[i]) {

                    result[index] = num;

                    index++;

                    if (index == k) {

                        break;

                    }

                }

            }

        }

        return result;

    }

}
```

Output:-

The screenshot displays the LeetCode submission interface for the problem "Top K Frequent Elements" (ID: 2142315827). The submission is marked as "Accepted" with 25/25 testcases passed, submitted by user "tameem_shaik" on Sep 15, 2026 at 12:23.

Performance Metrics:

- Runtime:** 11 ms | Beats 96.22%
- Memory:** 47.35 MB | Beats 91.38%

Code Editor: The code is written in Java. The visible snippet shows the end of a function:

```
26  
27  
28  
29     return result;  
30  
31 }
```

Testcase Results:

- Case 1:** Input: `nums = [1, 1, 1, 2, 2, 3]`, `k = 2`. Output: `[1, 2]`. Expected: `[1, 2]`.
- Case 2:** Passed.
- Case 3:** Passed.

The overall status is "Accepted" with a runtime of 0 ms.